sets $X \subseteq \mathcal{P}(V^+)$ of a transformer are sets of events of $\mathrm{Act}^{S5}(\mathcal{X})$ with matching preconditions. This means that nothing in the following chapters is the case just because we chose to use the symbolic setting, so whatever we prove symbolically is also a claim about S5 action models.

2.5 Symbolic bisimulations and static minimisation

When are two knowledge structures equivalent? As Gattinger observes when introducing the notion ([Gat18, §2.11]), the question carries a hidden parameter, the vocabulary for which we ask it. Two structures can encode their epistemic relations with entirely different auxiliary atoms and still describe the same situation about the atoms that we care about, so equivalence must be relative to a chosen part of the shared vocabulary. On the explicit side the matching notion is a bisimulation for a set of atoms U in the sense of Definition 4.12. On the symbolic side, relations between structures are boolean formulas, here over a doubled vocabulary.

For a set W of atoms let $W^* := \{w^* \mid w \in W\}$ be a fresh disjoint *starred* copy, and for $u \subseteq W$ let $u^* := \{w^* \mid w \in u\}$. A boolean formula over a doubled vocabulary then encodes a relation between two structures, the first read on the un-starred atoms and the second read on the starred atoms.

Definition 2.43 (Symbolic bisimulation [Gat18, Def. 2.11.1])**.** *Let $\mathcal{F}_1 = (V_1, \theta_1, O^1)$ and $\mathcal{F}_2 = (V_2, \theta_2, O^2)$ be knowledge structures and $V \subseteq V_1 \cap V_2$. To keep atoms that the structures share* outside *V apart, let $\overline{V} := V \cup \big((V_1 \setminus V) \uplus (V_2 \setminus V)\big)$, each $\mathcal{F}_\ell$ read over its injective image in $\overline{V}$; when $V_1 \setminus V$ and $V_2 \setminus V$ are already disjoint, $\overline{V}$ is simply $V_1 \cup V_2$ and nothing is renamed. A boolean formula $\beta \in \mathcal{L}_B(\overline{V} \cup \overline{V}^*)$ is a* symbolic bisimulation for *V between $\mathcal{F}_1$ and $\mathcal{F}_2$ iff for all states s_1 of $\mathcal{F}_1$ and s_2 of $\mathcal{F}_2$ with $s_1 \cup s_2^* \models \beta$:*

(1) agreement*: s_1 and s_2 agree on every atom of V;*

(2) forth*: for every agent j and every state t_1 of $\mathcal{F}_1$ with $s_1 \sim_j^{\mathcal{F}_1} t_1$ there is a state t_2 of $\mathcal{F}_2$ with $t_1 \cup t_2^* \models \beta$ and $s_2 \sim_j^{\mathcal{F}_2} t_2$;*

(3) back*: symmetrically.*

Scenes $(\mathcal{F}_1, s_1)$ and $(\mathcal{F}_2, s_2)$ are bisimilar for *V iff some symbolic bisimulation β for V has $s_1 \cup s_2^* \models \beta$.*

Remark 2.44 (Why the doubled leftovers)**.** *One might hope that $\bigwedge_{p \in V}(p \leftrightarrow p^*)$ is always a symbolic bisimulation for V between equivalent structures, but it is not. The reason for this lies in the role of $\overline{V}$. Semantic equivalence for the modal language $\mathcal{L}(V)$ does not need to be expressible over just $V \cup V^*$, because the epistemic relations of the two structures may be encoded by different atoms outside V, and β has to be allowed to mention those ([Gat18, §2.11]).*

We do not need to check the three conditions state by state, as each of them is a single boolean statement about β, the two laws, and the observables.

Fact 2.45 (Symbolic bisimulations are boolean [Gat18, Lem. 2.11.2])**.** *For knowledge structures $\mathcal{F}_1, \mathcal{F}_2$ and $\beta \in \mathcal{L}_B(\overline{V} \cup \overline{V}^*)$, the following are equivalent:*

(i) β *is a symbolic bisimulation for* V *between* $\mathcal{F}_1$ *and* $\mathcal{F}_2$*;*

(ii) β *encodes, via* $s_1 \cup s_2^* \models \beta$*, a bisimulation for* V *in the sense of Definition 4.12 between the models* $M(\mathcal{F}_1)$ *and* $M(\mathcal{F}_2)$*;*

(iii) three boolean formulas over the doubled vocabulary, one per condition of Definition 2.43, built from β*,* θ_1*,* θ_2^* *and boolean quantifiers over the observable complements, are tautologies.*

For the three displayed formulas of (iii) we refer to [Gat18, Lem. 2.11.2], which discusses that checking a candidate β *amounts to three BDD identities, no explicit model is built, and the forth and back conditions appear as quantifier prefixes that can also be run in reverse, as a refinement operator. Section 4.5 uses this to compute bisimulations.*

Fact 2.46 (Symbolic bisimulation transfers truth [Gat18, Thm. 2.11.3])**.** *If* β *is a symbolic bisimulation for* V *between* $\mathcal{F}_1$ *and* $\mathcal{F}_2$ *with* $s_1 \cup s_2^* \models \beta$*, then* $(\mathcal{F}_1, s_1) \models \varphi$ *iff* $(\mathcal{F}_2, s_2) \models \varphi$ *for every* $\varphi \in \mathcal{L}(V)$*.*

The converse holds as well, and since our structures are finite it holds without any side condition. This is the symbolic form of the Hennessy-Milner theorem.

Fact 2.47 (Symbolic Hennessy-Milner [Gat18, Thm. 2.11.4])**.** *Let* $\mathcal{F}_1, \mathcal{F}_2$ *be knowledge structures with vocabularies* V_1, V_2*, let* $U \subseteq V_1 \cap V_2$*, and* s_1, s_2 *states of* $\mathcal{F}_1, \mathcal{F}_2$ *such that* $(\mathcal{F}_1, s_1) \models \varphi$ *iff* $(\mathcal{F}_2, s_2) \models \varphi$ *for every* $\varphi \in \mathcal{L}(U)$*. Then there is a symbolic bisimulation* β *for* U *between* $\mathcal{F}_1$ *and* $\mathcal{F}_2$ *with* $s_1 \cup s_2^* \models \beta$*.*

A structure can be needlessly large in two ways [Gat18, §2.12]. The first way is through an atom that the law has already fixed.

Fact 2.48 (Determined atoms can be eliminated [Gat18, Lem. 2.12.2])**.** *Let* $\mathcal{F}$ *be a knowledge structure over* $V \cup \{p\}$ *with* $p \notin V$ *determined by the state law, i.e.* $\theta \to p$ *or* $\theta \to \neg p$ *is a tautology. Then for* $\mathcal{F}' := (V,\ \exists p\, \theta,\ O_1 \setminus \{p\}, \ldots, O_n \setminus \{p\})$ *and every state* s *of* $\mathcal{F}$ *we have* $(\mathcal{F}, s) \equiv_V (\mathcal{F}', s \setminus \{p\})$*, witnessed by the symbolic bisimulation* $\beta := \bigwedge\{q \leftrightarrow q^* \mid q \in V\}$*. ([Gat18] states and proves this for belief structures, the displayed knowledge-structure form is the special case used here, with the same witnessing bisimulation.)*

Determined atoms are common. They can arise when factual change overwrites old atoms like in [Gat18, Ex. 2.12.1], and, in the setting of Chapter 3, when iterated updates leave behind choice bits for branches that never occur. The question Chapter 5 concerns itself with, namely when an event atom can be removed, is the dynamic version of this fact.

The second way in which a structure can be unnecessarily large comes from the law and the epistemic part. The encoded relations may repeat information that is already encoded by the law. An example is

$$(\{p,q\},\ p \leftrightarrow q,\ O_a = \{p,q\}) \quad \equiv_{\{p,q\}} \quad (\{p,q\},\ p \leftrightarrow q,\ O_a = \{p\}),$$

since, under the law, observing p is the same as observing q ([Gat18, Ex. 2.12.3]). In the general belief-structure setting, observation laws can thus be replaced by any equivalent law modulo the state law ([Gat18, Lem. 2.12.4]).

These optimisations have two important properties. They are safe, meaning that they only restrict diagrams against the law or delete atoms, so it will not increase the size of a BDD. They are also dual to the explicit-side optimisation of generated submodels, which incorporates reachability into the law and can make diagrams larger. This is why Gattinger avoids that optimisation ([Gat18, §2.12]). In Chapter 5 we will adopt both the safety requirement and the reluctance about the second optimisation.

All of this is *static* minimisation. It shrinks one structure while preserving its scenes up to $\equiv_V$. In the following chapters, however, we will ask the dynamic question of whether, given an event, there is a smaller event with the same effect on every structure. That question only makes sense once "same effect" has been defined and can be recognised, this is what we do inChapter 4. In Chapter 5 we then find a cheaper approximation of it.

2.6 Related work

This section places the contributions of the later chapters in the existing literature.

Symbolic model checking of knowledge. Symbolic model checking with BDDs starts from Bryant's work [Bry86] and from using diagram-based verification for large transition systems by Burch, Clarke, McMillan, Dill and Hwang [Bur+92], followed by McMillan's SMV [McM93] and Clarke, Grumberg and Peleg's account in [CGP99]. For knowledge, BDD-based approaches appear in temporal-epistemic model checkers like MCK [GM04], MCTK[2] [SSL07] and MCMAS [LQR09; LQR17]. All three work with interpreted systems, where knowledge is given by the agreement of local states and evolution is given by a transition relation. They do not offer a DEL-style product update. On the DEL side, the standard tools that have been developed are explicit: DEMO [Eij07] and DEMO-S5 [Eij14] use Kripke and action models. These two lines are combined in SMCDEL, where knowledge structures and transformers [Ben+15; Ben+18; Gat18] bring the product update into the BDD setting, and Chapter 2 of [Gat18] provides the underlying theory. In this setting, earlier symbolic work, including SMCDEL, evaluates a fixed update, or a fixed sequence of updates, on a structure. Based on this background, this thesis studies updates themselves as objects of a symbolic framework, with operations (union, composition, iteration; Chapter 3), an equivalence notion (Chapter 4), and an optimisation procedure (Chapter 5), which are all developed to extend the SMCDEL infrastructure developed by [Gat18]. Two further strands, namely variable forgetting in knowledge representation [LR94; LLM03] and symbolic bisimulation quotienting in verification [BS92; Wim+06], are relevant for the constructions in Chapter 5.

Update equivalence, action emulation, and minimisation. On the explicit side, the question about when two action models have the same update effect goes back to Ruan [Rua04], who raised the problem, observed that bisimilarity of action models is sufficient but not necessary for update equivalence, and proposed an emulation notion for the propositional case. Sadzik [Sad06] extended the analysis to iterated updates. Van Eijck, Ruan and Sadzik [ERS12]

[2]Which uses the same translation of K_j into $\forall(V \setminus O_j)$ as Definition 2.22, as noted in [Gat18, §3.1]

introduced *action emulation*, a structural relation weaker than bisimilarity that still implies update equivalence and characterises it for action models with propositional preconditions. A parameterised variant of this characterises update equivalence in general, with the cost of quantifying over formulas inside the relation. Sietsma and van Eijck [SE13] showed that update equivalence can be decided by using canonical action models, which can be obtained by strengthening every precondition with fresh atoms naming the actions, between which the emulation is complete. Since every action model is update equivalent to a canonical one, this gives a decision method for the general case. This is also discussed in [DHK08, §6.5]. A preprint by Li [Li22] proposes a generalised emulation that matches update equivalence exactly, recovers the earlier relations as special cases, and gives complexity bounds for minimisation on the explicit side. They state that shrinking the event space of an action model is PSPACE-complete under propositional action emulation and PSPACE-hard under update equivalence. The following chapters differ from that work in three respects. First, the discussed work concerns arbitrary Kripke models, so its update equivalence is in finer than one that only S5 models can refute, and we can not directly transfer the results. Second, it fixes one countable vocabulary, so the choice of test vocabularies and test formulas, which matters in our setting (as will be discussed in Section 4.2), is not relevant there. Third, it is set in the explicit setting, while this thesis concerns the explicit setting: an action model is measured in events, a transformer in atoms, and $2^{|V^+|}$ events fit into $|V^+|$ atoms, so even the notion of "smaller" changes, as Section 5.2 discusses. There is also a parallel between the two approaches: Sietsma and van Eijck [SE13] note a vocabulary cost on the action side, with fresh atoms naming events, while we will need fresh atoms naming epistemic profiles in Section 4.6, which gives us an analogous cost on the static side.

Iterating updates. What happens when an update may be applied repeatedly? On the explicit side, Miller and Moss [MM05] showed that iterated modal relativisation and adding a Kleene star for announcements to modal logic makes the satisfiability problem undecidable. Quantifying over announcements instead of iterating them does not change this. Arbitrary public announcement logic, with $\Diamond\varphi$ read as "some announcement makes φ true", was introduced by Balbiani, Baltag, van Ditmarsch, Herzig, Hoshi and de Lima [Bal+08], and its satisfiability problem was shown to be undecidable by French and van Ditmarsch [FD08]. Ågotnes, van Ditmarsch and French [ÅDF16] extended this to group and coalition announcement logic, that is decidable only for the single-agent case. While these results are about satisfiability, model checking for the same logics is decidable, and in this thesis we will focus on model-checking, where every search for a formula to hold after a fixed number of iterations of a transformer is one boolean computation (Section 3.4) but checking at the unbounded star is difficult. Further, Section 4.9 discusses how to detect when an iteration has converged, but it does not claim that convergence always occurs. This section is related to the update evolution in [Rua04] and the iterated update universe from [Sad06], which study stabilisation of repeated updates in the explicit framework.

Epistemic planning. We use epistemic planning results throughout this thesis. Bolander and Andersen [BA11] treat planning as reachability by product updates from a belief state (a set of worlds the planner still considers possible) and show that plan existence is undecidable

in the multi-agent case. Aucher and Bolander [AB13] strengthen this by showing that this undecidability persists even under strong restrictions on the events. Andersen, Bolander and Jensen [ABJ12] distinguish strong plans, which reach the goal under every contingency, from weak plans, which merely can reach it, for conditional planning in DEL. In classical, non-epistemic planning, the same distinction has already been used for BDD-based symbolic model checking in work by Cimatti, Pistore, Roveri and Traverso [Cim+03], so the introduction of universal and existential readings of belief states in Section 3.1 is an epistemic instance of this established distinction. The four readings of Theorem 3.12 verify a given multi-pointed action against a belief state, which is decidable and inexpensive, while synthesis (existence of a plan) reverses the quantifiers, this is undecidable in general. Bolander's survey [Bol17] gives ab overview and an introduction to epistemic planning, and SMCDEL-based planning experiments appear in [Gat18, §4.8].

Choice, composition, iteration. The operations we will introduce in Chapter 3 are DEL instances of operators from PDL. PDL closes programs under choice, composition and star [FL79; HKT00], and Kleene algebra gives us the equational theory of these operations [Koz94]. On action models, choice and composition behave as in PDL: $[\alpha \cup \beta]\varphi \leftrightarrow [\alpha]\varphi \wedge [\beta]\varphi$, and composing this to an iterated product is done also on Baltag and Moss's epistemic programs [BM04] and in [DHK08, Props. 6.10–6.11]. Corollary 3.45 and Corollary 4.50 show that multi-pointed events are a monoid under composition, first up to applied-structure equality and then up to equivalence. The star is where DEL diverges from PDL. Iteration of updates has no reduction axioms, and its logic is the undecidable one that we discussed the previous paragraph. This thesis therefore treats iteration semantically rather than axiomatically, by bounded horizons (Section 3.4), a convergence test (Section 4.9), and a pruned fixpoint (Section 5.6).

The following chapters will assume the material that we introduced above. The facts are annotated with the numbers under which they are proved in [Gat18]. The lemmas Lemma 2.14, Lemma 2.33, Lemma 2.38, Lemma 4.13, Lemma 2.39 that we proved here will be used in the next chapters. We now have two formats, the explicit one and the symbolic one, as well as a fixed translation between them. The next chapter combines several transformers.

Chapter 3

Uncertainty and Combination of Knowledge Transformers

We have seen how a knowledge structure can be updated by a single knowledge transformer and how the dynamic operator $[\mathcal{X}, x]$ that this induces reduces, like the static operators we introduced before, to a boolean computation. But all of that describes one update, by one event, evaluated at one actual world. In this chapter we put several such updates together. We define the *union* of m alternative transformers, which models a single non-deterministic choice, their *composition*, which models doing one update and then another, and the *iteration* of a non-deterministic choice over many rounds. Throughout we work in a *multi-pointed* setting, which we read as the two uncertainties that a planner faces: uncertainty about *which outcome* an action produces, and uncertainty about *which world* is actual. The first uncertainty concerns events. An update does not need to have a single designated outcome. A non-deterministic action ("one of these m things happens, but we do not say which") designates a whole set X of candidate events. This is the reading of a multi-pointed action model, where we have a non-deterministic choice between single-pointed alternatives, whose non-determinism is the direct sum of those alternatives [BM04] [DHK08, §§ 6.3–6.4, § 6.10]. The disjoint union we build in Section 3.2 is this sum. The second uncertainty concerns the state at which we evaluate, which does not need to be a single world. A planner reasons from a *belief state*, which is a set S of worlds that are, as far as it can tell, all still possible ([BA11]). A goal is achieved at such a state when it holds, no matter which of these worlds turns out to be the actual one.

We describe each designated set by a boolean *characteristic formula* (as introduced by Definition 2.16 and **??**): a set S of worlds by a formula σ_S over the structure vocabulary V, a set X of events by a formula ξ_X over the event vocabulary V^+. Since V and V^+ are disjoint, these two formulas can never have the same atom and we can thus combine them by a conjunction. They differ only in their quantifiers, as a set of actual events records the non-determinism of the action, which one reads existentially ("after *some* actual event"), whereas a set of actual worlds records uncertainty about the state, which one reads universally ("in *every* world still possible"). Combining the two gives four readings of "φ holds after the update", the two with worlds read universally are the strong and weak notions of conformant and contingent planning ([ABJ12]). All four, as we will see, reduce to one boolean computation that is guarded by ξ_X and σ_S.

We set up this framework for an arbitrary transformer in Section 3.1, and then construct

the union (Section 3.2), the composition (Section 3.3), and the iteration (Section 3.4). Each construction yields a transformer together with a designated event set, and the tools from the multi-pointed framework then provide both the symbolic evaluation and the lift to belief states. What remains in each case is to show that the construction is *correct*, i.e. that it really computes what it should, and this is proved, in each section, by a truth-preservation theorem. Throughout this chapter we illustrate the constructions on the following scenario.

Example 3.1 (The running example: a surprise party)**.** *Malvin's birthday is coming up, and Johan may or may not have made up his mind to throw a surprise party for him. Three friends, Alice, Bob, and Charlie (a, b, c) are waiting to hear. With p for "the party is on" and d for "Johan has decided whether he will throw the party", let*

$$\mathcal{F}_0 = (\{p, d\},\ p \to d,\ O_a{=}\emptyset,\ O_b{=}\emptyset,\ O_c{=}\emptyset).$$

The state law rules out a party that is on before anyone decided to throw it, leaving three states: $\emptyset$ (still undecided), $\{d\}$ (decided against), $\{p, d\}$ (it is on). Nobody observes anything, so no friend knows whether there even is a plan yet. For each friend there is one transformer in which Johan calls the other two *and tells them whether the party is on. Johan will never call all three, because he leaves out whichever friend he currently judges likeliest to let something slip, and he will not call before his mind is made up. Writing the transformers with event vocabularies $V_1^+ = \{q_1\}$, $V_2^+ = \{q_2\}$, $V_3^+ = \{q_3\}$,*

$$\mathcal{X}_1 = (\{q_1\},\ d \wedge (q_1 \leftrightarrow p),\ O^+_{1,a}{=}\{q_1\},\ O^+_{1,b}{=}\{q_1\},\ O^+_{1,c}{=}\emptyset),$$

and symmetrically $\mathcal{X}_2 = (\{q_2\},\, d \wedge (q_2 \leftrightarrow p), \dots)$ with audience $\{a, c\}$ (so $O^+_{2,b} = \emptyset$) and $\mathcal{X}_3 = (\{q_3\},\, d \wedge (q_3 \leftrightarrow p), \dots)$ with audience $\{b, c\}$ (so $O^+_{3,a} = \emptyset$). Each $\mathcal{X}_i$ is a semi-private announcement with a precondition: the event atom q_i records the announced plan through $q_i \leftrightarrow p$, exactly the two friends in the audience observe q_i, and the conjunct d makes the call impossible while Johan is still mulling it over. Applying $\mathcal{X}_1$ to $\mathcal{F}_0$ leaves the two states $\{d\}$ and $\{p, d, q_1\}$: the undecided world survives no event, so everyone learns d from the fact that calls went out, while only the audience learns p. As discussed after Definition 2.20, the occurrence of the action is common knowledge, only the content is semi-private, so even c, who is told nothing, knows that calls went out, and hence that Johan has made up his mind.

3.1 Multi-pointed scenes and events

We set up the two multi-pointings. We do this for an arbitrary but fixed structure $\mathcal{F} = (V, \theta, O_1, \dots O_n)$[1] and transformer $\mathcal{X} = (V^+, \theta^+, O_1^+, \dots, O_n^+)$ with V^+ disjoint from V, so that we can instantiate it in the later sections. A designated set S of worlds is recorded by $\sigma_S \in \mathcal{L}_B(V)$, and a designated set X of events by $\xi_X \in \mathcal{L}_B(V^+)$ (Definition 2.16 and **????**). There are two boundary cases. A singleton fixes every atom of its vocabulary, that is, $\xi_{\{x\}} = (x \sqsubseteq V^+)$, and likewise $\sigma_{\{s\}} = (s \sqsubseteq V)$. For the full sets, $X = \mathcal{P}(V^+)$ gives $\xi_X \equiv \top$, and $S = \mathrm{States}(\mathcal{F})$

[1]Nothing in this section constrains V, θ or the O_j, so a result proved about $\mathcal{F}$ holds for every knowledge structure, and we apply such results to updated structures such as $\mathcal{F} \times \mathcal{X}$ without further comment.